

Information distillation from physical simulation

Damian Gieriek

07.02.2025

In this paper, I use the method of unsupervised learning of equations of motion for objects in raw and optionally distorted unlabelled synthetic video, presented in this article [1], for the case of videos of bodies moving in gravitational field to obtain the trajectories of the bodies. To obtain this, I trained an autoencoder that maps each video frame into a low-dimensional latent space that allows reproducing the original image. The desired 2D Cartesian trajectory is obtained via principal component analysis of vectors from the latent space. The paper presents preliminary results.

1 Motivation and methodology

First, let us clarify why this approach is interesting. By obtaining the trajectory of a moving body from a video, we can guess the formula that governs the movement of the body using symbolic regression. An example of an algorithm that allows it is "AI Feynman" [2]. This approach was analogous to the work of Johannes Kepler, who discovered that Mars' orbit is an ellipse after analysing the data of its orbit [3].

The method is based on training a specific architecture consisting of an autoencoder and an evolution operator performed on a set of snapshots of a simulation of a body moving in a gravity field.

2 Code library

The whole code I used for the project is publicly available here: https://github.com/dgieriek/physics_guesser.

The library contains the module "data_generator.py" that is responsible for generating simulation data. The module uses another module called "rocket_simulation.py" that stores the analytical solutions used in simulation generation. Simulations are stored in the file "simulation_frames".

The module called "autoencoder.py" contains an implementation of used neural network architecture. Whilst the module "datasets.py" contains data loaders that inherit from torch.utils.data.Dataset and load the simulation data during training sessions. The training loop is implemented in the module called "trainer.py".

Finally, in the "evaluator.py" module I implemented the method for calculating the loss function. The "evaluation.py" module stores the implementation of principal component analysis and visualisation of images and trajectories reconstructed by the neural network.

2.1 Simulation of a body moving in a gravity field

The first problem to solve for the project was to create a set of data with which the neural network will be trained. In the beginning I decided to try to simulate the movement in a gravity field using numerical integration of differential equations of motion via the Verlet method. Then,

I realised it is not necessary as using the analytical solutions is sufficient. In the end, I created 145 simulations with random starting positions, velocities, and directions of the gravity acceleration vector. Each simulation contained about 30 snapshots/frames. Such an example frame is shown in figure 1. An example trajectory of one of the simulations was shown in graph in figure 2.

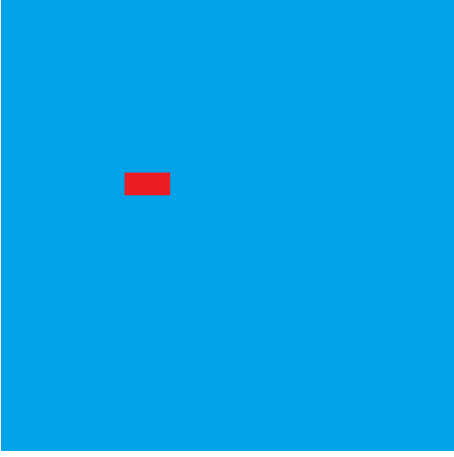


Figure 1: An example snapshot of one of the simulations of a body (the red rectangle) moving in a gravity field on the blue background.

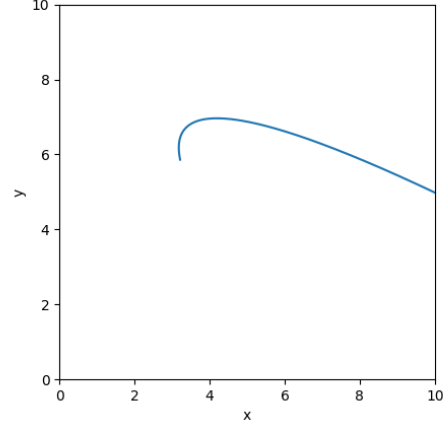


Figure 2: The graph shows the 2D trajectory of the body moving in a gravity field from an example simulation.

2.2 Neural network architecture

The neural network architecture consists of autoencoder (encoder and decoder) and an evolution operator. The encoder consists of five convolutional ReLU layers (`torch.nn.Conv2d` and `torch.nn.ReLU` from PyTorch library) with kernel size 4 and padding 1, four of them with stride 2 followed by one with stride 1. At the end there is a fully connected linear layer (`torch.nn.Linear`) that gives a latent space vector of dimension 10. The number of channels goes from 3 for the input image to 32, 32, 64, 64 and 256 for the convolutional layers. The decoder network is a mirror image of the encoder in terms of layer dimensions, with the convolution layers replaced by deconvolution layers (`torch.nn.ConvTranspose2d`). The evolution operator has three fully connected 32-neuron layers (`torch.nn.Linear`) with softplus activation function (`torch.nn.Softplus`) and a linear 10-neuron output layer. The evolution operator takes two vectors from latent space (the vector from the current and the previous time step) and predicts the latent space vector for the next time step. A graphical illustration of the architecture is shown in figure 3.

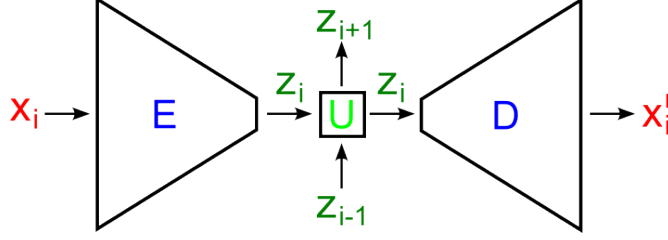


Figure 3: A draft of a neural network architecture consisting of encoder - E, decoder - D and an evolution operator - U. Encoder E encodes a snapshot from a video (a frame of a video with the moving body) - x_i into a low dimensional latent space vector z_i . The evolution operator U takes the latent space vector z_i and the latent space vector from a previous simulation time step - z_{i-1} (we receive it by encoding x_{i-1}), and returns the prediction of latent space vector of the following time step z_{i+1} . Then, the decoder D decodes the latent space vector z_i and returns the reconstructed snapshot from the video - x_i^r .

2.3 Loss function

The loss function [1] was implemented as follows:

$$\mathcal{L} = \mathcal{L}^{recon} + \alpha \mathcal{L}^{pred} + \beta \mathcal{L}^{nl} + \gamma \mathcal{L}^{acc}.$$

These four terms are averages over all time steps i of the following functions:

$$\begin{aligned}\mathcal{L}_i^{recon} &= \frac{|x_i - D(E(x_i))|}{|x_i|}, \\ \mathcal{L}_i^{pred} &= \frac{|z_{i+1} - U(z_i, z_{i-1})|}{|z_i - z_{i-1}|}, \\ \mathcal{L}_i^{nl} &= \frac{1}{4000} |z_i - z_{i-1}| \cdot \|\nabla \mathbf{J}(\mathbf{w}_i)\|_1, \\ \mathcal{L}_i^{acc} &= \frac{1}{10} \|U(\mathbf{w}_i) - \mathbf{M}\mathbf{w}_i\|_1,\end{aligned}$$

where α, β, γ are tunable hyperparameters, $\mathbf{w}_i = \begin{pmatrix} z_{i-1} \\ z_i \end{pmatrix}$, $\mathbf{J}$ is the Jacobian of an evolution operator U, $\mathbf{M} = \begin{pmatrix} -\mathbf{I} & 2\mathbf{I} \end{pmatrix}$ and $\mathbf{I}$ is the 10x10 identity matrix.

3 Preliminary results

After performing a training session on all the simulations of a body moving in a gravity field I realised a big flaw in my training loop. Namely, the loss function was not calculated as described in subsection 2.3 as an average of all time steps in one simulation but after each frame. Although the components of the loss function appeared to get smaller during training session, after visualizing the reconstructed images of snapshots of a simulation and comparing them with original images, it is clear that something is wrong. The graph of the values of loss function and its components with respect to the training epoch is presented in figure 4. The comparison between an original image and the reconstructed one is shown in figure 5.

I have also created an image of a reconstructed trajectory. By comparing it with the original trajectory, one could think it seems to be almost correct (neglecting some rotations and linear scaling). However, in my opinion, since I used flawed training loop, it is rather a coincidence. The comparison between an original trajectory with the reconstructed one is shown in figure 6.

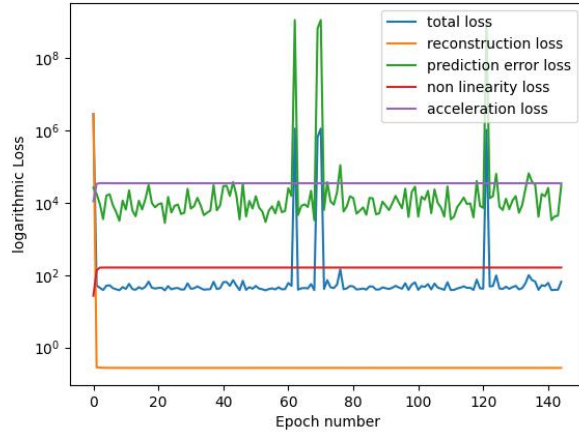


Figure 4: Components of the loss function versus epoch number during training. Each epoch corresponds to training performed on one simulation.

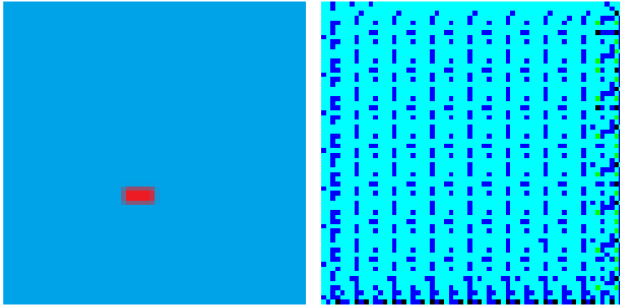


Figure 5: The comparison between original snapshot of a simulation (the photo is blurred as the images entering encoder are deliberately downsampled) on the left and the reconstructed image on the right. It is clear that reconstruction went wrong.

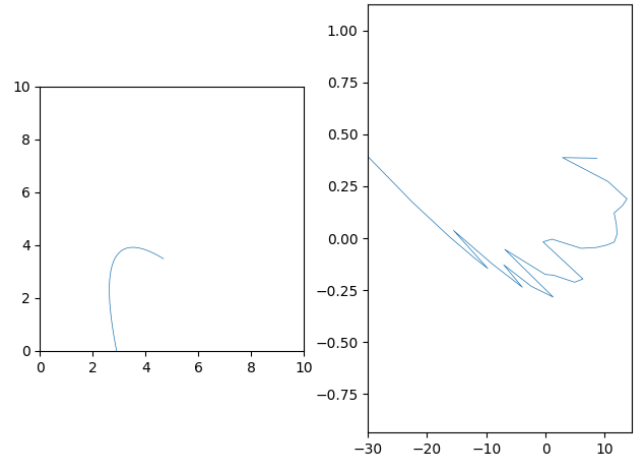


Figure 6: The comparison between an original trajectory of a body on the left with the reconstructed trajectory on the left.

4 Summary and future work

In the paper I presented the preliminary results of my attempt to reproduce the results of the article [1]. For now, I did not manage to obtain a correctly working algorithm that reconstructs

the trajectory of a moving object from a video due to a flawed training loop procedure.

In the future work, I will concentrate on repairing the issue with training loop and try to obtain the desired results. The work that is still ahead is implementing a comprehensive evaluator that would assess the quality of latent space trajectory representations and extending the training of the model on other types of motion like movement in magnetic field, harmonic oscillator. In the end it would be interesting to see, if the neural network architecture would be able to detect the trajectories of bodies from real footages.

References

- [1] Silviu-Marian Udrescu and Max Tegmark. “Symbolic pregression: Discovering physical laws from distorted video”. In: *Phys. Rev. E* 103 (4 Apr. 2021), p. 043307. DOI: 10.1103/PhysRevE.103.043307. URL: <https://link.aps.org/doi/10.1103/PhysRevE.103.043307>.
- [2] Silviu-Marian Udrescu and Max Tegmark. *AI Feynman: a Physics-Inspired Method for Symbolic Regression*. 2020. arXiv: 1905.11481 [physics.comp-ph]. URL: <https://arxiv.org/abs/1905.11481>.
- [3] Alexandre Koyré. *The Astronomical Revolution: Copernicus-Kepler-Borelli*. Routledge, 2013.